==========

==========

CHAPTER 8 — TLB ARCHITECTURE AND BEHAVIOR

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8 TLB architecture, entry structure, tagging, coherence, walk caching, invalidation, and hypervisor interactions.

****

****

8.1 — Introduction to the TLB

****

****

The **Translation Lookaside Buffer (TLB)** is a hardware cache that stores recently used address translations. It accelerates memory access by avoiding page table walks.

TLB entries contain:

- virtual address (VA)
- intermediate physical address (IPA)
- physical address (PA)
- ASID (process ID)
- VMID (VM ID)
- permissions
- memory attributes
- shareability
- global/non-global flag

TLB behavior is central to MMU performance and correctness.

****

****

8.2 — Why the TLB Exists

****

****

Page table walks are expensive:

- multiple memory accesses
- descriptor interpretation
- potential Stage-1 + Stage-2 walk
- cache misses
- walk cache misses

TLB avoids these by caching final translations.

Teaching Note

TLB hits are critical for performance. TLB misses can degrade performance significantly.

****

****

8.3 — Types of TLBs in ARMv8

****

****

ARMv8 typically implements:

Instruction TLB (ITLB)

Caches instruction fetch translations.

Data TLB (DTLB)

Caches data access translations.

Unified TLB (UTLB)

Caches both instruction and data translations.

L1 TLB

Small, fast, per-core.

L2 TLB

Larger, slower, shared across cores or clusters.

Implementation Note

Exact TLB structure varies by microarchitecture (Cortex-A53 vs A72 vs Neoverse).

****

****

8.4 — TLB Entry Structure

****

****

A TLB entry contains:

```
VA tag
ASID
VMID
IPA or PA
permissions (AP/S2AP)
execute permissions (PXN/UXN/S2PXN/S2UXN)
memory attributes (AttrIndx → MAIR)
shareability (SH/S2SH)
global bit
valid bit
```

Teaching Note

TLB entries store **final merged permissions and attributes**.

****

****

8.5 — TLB Tagging: ASID + VMID

****

****

TLB entries are tagged with:

- **ASID** → process isolation
- **VMID** → VM isolation

Combined Tag

```
TLB_tag = (VA, ASID, VMID)
```

This allows:

- multiple processes
- inside multiple VMs
- to coexist in the TLB
- without flushing on context switches or VM switches

****

****

8.6 — Global vs Non-Global TLB Entries

****

****

Global Entries

- belong to TTBR1_EL1 (kernel)
- shared across all ASIDs
- not flushed on context switch

Non-Global Entries

- belong to TTBR0_EL1 (user)
- tagged with ASID
- flushed when ASID reused

Teaching Note

Kernel mappings should always be global for performance.

****

****

8.7 — TLB Lookup Process

****

****

TLB lookup requires:

```
VA + ASID + VMID
```

Steps:

1. Extract VA tag
2. Determine TTBR region
3. Match ASID
4. Match VMID
5. Check permissions
6. Check attributes
7. Return IPA/PA

If any mismatch → TLB miss.

****

****

8.8 — TLB Miss Handling

****

****

TLB miss triggers:

1. Stage-1 walk
2. Stage-2 walk
3. Merge permissions
4. Merge attributes
5. Insert into TLB

Implementation Note

Walk cache accelerates multi-level walks.

****

****

8.9 — Walk Cache

****

****

Walk cache stores:

- intermediate descriptors
- L1/L2 table pointers
- partial walk results

Benefits:

- avoids reloading table descriptors
- reduces memory traffic
- accelerates page table walks

Teaching Note

Walk cache is separate from TLB.

****

****

8.10 — TLB Coherence

****

****

TLB coherence ensures:

- all cores see consistent translations
- invalidations propagate correctly
- no stale entries remain

ARMv8 uses:

- broadcast TLBI
- inner shareable domain
- outer shareable domain

Implementation Note

TLB coherence is critical for SMP correctness.

****

****

8.11 — TLB Invalidation (TLBI)

****

****

TLBI instructions invalidate TLB entries.

Examples:

- **TLBI VAE1** → invalidate by VA + ASID
- **TLBI ASIDE1** → invalidate all entries for ASID
- **TLBI VMALLE1** → invalidate all Stage-1 entries
- **TLBI ALLE1** → invalidate all entries

Teaching Note

TLBI must be followed by DSB + ISB.

****

****

8.12 — TLB Invalidation Sequence

****

****

Correct sequence:

1. Write descriptor
2. DSB SY
3. TLBI instruction
4. DSB SY
5. ISB

Implementation Note

Skipping barriers causes silent corruption.

****

****

8.13 — TLB and Break-Before-Make (BBM)

****

****

BBM rule:

Invalidate old descriptor before installing new descriptor.

Required when changing:

- block → page
- page → block
- block → table
- table → block
- permissions
- memory attributes
- physical address

Teaching Note

BBM prevents TLB aliasing and undefined behavior.

****

****

8.14 — TLB and Speculation

****

****

Speculative execution may:

- fill TLB entries
- perform speculative walks
- use stale descriptors

Implementation Note

Barriers prevent speculative use of stale mappings.

****

****

8.15 — TLB and Hypervisor Interaction

****

****

Hypervisor controls:

- VMID allocation
- Stage-2 TLB invalidation
- nested TLB invalidation
- shadow page tables

Teaching Note

Hypervisor must flush Stage-2 TLB entries when remapping IPA → PA.

****

****

8.16 — TLB and Nested Virtualization

****

****

Nested virtualization requires:

- nested VMID
- nested ASID
- nested TLBI
- shadow Stage-2 tables
- shadow Stage-1 tables

Implementation Note

Nested TLB invalidation is complex and performance-critical.

****

****

8.17 — TLB and Device Memory

****

****

Device memory mappings:

- must be non-cacheable
- must be strongly ordered
- must not be speculated
- must not be prefetched

TLB entries for device memory:

- contain Device attributes
- enforce ordering
- disable speculation

****

****

8.18 — TLB and Memory Partitioning (QNX-Style)

****

****

QNX uses:

- ASID ranges for partitions
- VMID ranges for VMs
- global kernel mappings
- per-partition user mappings

TLB behavior aligns with partition boundaries.

****

****

8.19 — TLB Fault Conditions

****

****

Faults occur if:

- TLB entry is stale
- permissions mismatch
- attributes mismatch
- ASID mismatch
- VMID mismatch
- descriptor invalid

Fault types:

- Permission Fault
- Execute Fault
- Translation Fault

****

****

8.20 — Chapter Summary

****

****

This chapter covered:

- TLB architecture
- entry structure
- ASID + VMID tagging
- global vs non-global entries
- lookup process
- miss handling
- walk cache
- coherence
- TLBI
- BBM

- speculation
- hypervisor interaction
- nested virtualization
- device memory
- QNX partitioning
- fault conditions